Session 03 — Functions

Python for Everyone · Lecture notes

> **How to read the code boxes:** grey boxes with a yellow bar contain Python code. The box labelled **Output** next to it (or below it) shows what Python prints when you run that code.

Functions

- Much of Python's functionality comes as **functions**. To **call** a function: `name_of_function(argument_1, argument_2)`.
- A **parameter** is an input that a function expects; the value you give it in a call is an **argument**. Many people use the two terms interchangeably.
- `print` displays its input on the screen and returns nothing. `len` **returns** the length of a string:

```
s = 'supercalifragilisticexpialidocious'
print(len(s))
```

Output:

```
34
```

User defined functions

- Functions let you **re-use code** instead of typing it again, and organize your code into logical units. This makes it easier to read, write and debug.
- `input`, `len` and `print` are **built-in**: they come with Python. `def` defines your own function:

```
def echo(message):
    print(message)
    print(message)

echo("What's up?")
```

Output:

```
What's up?
What's up?
```

- The `def` line ends with a colon `:`, and the **indented** block is the body of the function.
- `def` only **defines** the function: nothing runs until you **call** it.

Return value

- `input` and `len` **return** values, which you can save in a variable and use later. `print` and `echo` only display something.
- Your own functions return values with the `return` statement:

```
def double_return(x):
    double = 2*x
    return double

d = double_return(12)
print(d)
```

Output:

```
24
```

- Compare with a function that only displays the result. An empty `return` ends the function without returning anything, and it is optional:

```
def double_print(x):
    double = 2*x
    print(double)
    return

double_print(21)
```

Output:

```
42
```

- A function can have several `return` statements. It stops at the first one it reaches, so only one of them runs:

```
def absolute_value(x):
    if x<0:
        return -x
    else:
        return x

print(absolute_value(-3), absolute_value(3))
```

Output:

```
3 3
```

- When you write a function, always test it using several inputs.

Passing multiple arguments

```
def check_divider(check_total, tip_percentage, num_people):
    # Increase the total with the tip
    total_to_pay = check_total * (1. + tip_percentage)
    # Divide equally among the party
    amount_a_pop = total_to_pay / num_people
    return amount_a_pop
```

- **Positional:** list the arguments in the order of the definition.
- **Keyword:** name the arguments. They can be in any order, and the code is very explicit, which can make it easier to understand.
- **Mixed:** positional arguments first, then keywords.

```
print(check_divider(10, .2, 2))                                    # positional
print(check_divider(check_total=10, tip_percentage=.2, num_people=2))  # keyword
print(check_divider(check_total=10, num_people=2, tip_percentage=.2))  # any order
print(check_divider(10, num_people=2, tip_percentage=.2))          # mixed
```

Output:

```
6.0
6.0
6.0
6.0
```

- Once you switched to keywords, you cannot go back to positional:

```
print(check_divider(check_total=10, .2, 2))
```

Output:

```
SyntaxError: positional argument follows keyword argument
```

Functions within functions

- You can build on your own functions by calling them inside other functions:

```
def max_of_two(x, y):
    if x>y:
        return x
    else:
        return y

def max_of_three(x, y, z):
    m2 = max_of_two(x, y)    # get maximum of first two
    m3 = max_of_two(m2, z)   # compare to third
    return m3

print(max_of_three(1, 13, 2))
```

Output:

```
13
```

Local and global variables

- Variables created inside a function are **local**: they exist only while the function runs and cannot be used outside it. So names inside a function do not collide with names outside.

```
def longer(word1, word2):
    length1 = len(word1)
    length2 = len(word2)
    if length1>length2:
        print(word1, "is longer")
    else:
        print(word2, "is longer")

longer("Dude", "Duderino")
print(length1)
```

Output:

```
Duderino is longer
NameError: name 'length1' is not
defined
```

- Variables defined outside functions are **global**: they are visible from anywhere, even inside functions:

```
def print_sum():
    print(x1+x2)

x1 = 2
x2 = 3
print_sum()
```

Output:

```
5
```

- This works, but it is **not recommended**. Pass all the data a function needs as parameters: it stays clear what the function relies on, which makes the code more readable and less error prone.
- If a local and a global variable have the same name, Python uses the local one. It looks for a global variable only if there is no local one.

Modules

- Most of Python's utility comes in **modules** that you have to import: plotting, data cleaning, statistics, machine learning, opening a pdf, … You can build on the work of the community.
- `math` comes with Python and contains common mathematical functions. Its documentation lists them all.

```
import math
print(math.sqrt(2), math.pi)
```

Output:

```
1.4142135623730951 3.141592653589793
```

- `math.pi` is not a function, but a float variable.
- Other ways to import:
  - `import math as ma`: use an abbreviation, e.g. `ma.sqrt(2)`.
  - `from math import sqrt, pi`: use the given names directly, e.g. `sqrt(2)`.
  - `from math import *`: import everything to use directly. Two modules might have a function with the same name, and it is harder to keep track of where your functions come from.

Common mistakes

- **Defining a function without calling it:** after `def echo(message):` nothing happens until you write `echo("Hi")`.
- **Printing instead of returning:** `d = double_print(21)` displays `42`, but `d` has no useful value: `print(d)` displays `None`. Use `return` if you need the result later.
- **Positional arguments in the wrong order:** no error, just a wrong result: `check_divider(10, .2, 2)` gives `6.0`, but `check_divider(10, 2, .2)` gives `150.0`. Keyword arguments avoid this.
- **Code after `return`:** the function stops at `return`, so the lines below it in the same block never run.
- **Using a global instead of the parameter:** the function below should print `person`, but it prints the global variable `name`. There is no error, so it is easy to miss: every call greets Vera. Test with several inputs.

  ```
  name = "Vera"
  def greet(person):
      print("Hello", name)   # should be person
  greet("Ilka")              # prints: Hello Vera
  ```

- **Forgetting to import:** `math.sqrt(2)` without `import math` gives `NameError: name 'math' is not defined`.

Summary

Term / code	Meaning
`name(a, b)`	call a function with the arguments `a` and `b`
parameter / argument	an input a function expects / the value given to it
built-in function	comes with Python: `print`, `input`, `len`, …
`def name(x):`	define your own function; the indented block is its body
`return value`	end the function and give back a value
positional / keyword	`f(1, 2)` / `f(a=1, b=2)`
local variable	created in a function, only exists inside it
global variable	defined outside functions, visible from anywhere
`import math`	make a module available: `math.sqrt(2)`
`import math as ma`	same, with an abbreviation: `ma.sqrt(2)`
`from math import sqrt`	use a name directly: `sqrt(2)`
`NameError`	this name is not defined here